

무인 전차 자율주행 시스템 기획 및 기술 명세서

1. 서론 (Introduction)

본 문서는 Unity 기반 전차 시뮬레이터와 ROS2 프레임워크가 결합된 **전차 자율주행 및 인지 시스템**의 전체 아키텍처와 작동 원리를 기술한 보고서입니다. 단순한 A to B 주행을 넘어, 적의 위협을 회피하고 협지를 돌파하는 자율주행 알고리즘을 목표로 합니다.

보고서 전반에 걸쳐 단순한 '기능 나열'에 그치지 않고, **왜(Why) 그런 방식을 택했는지**, 그리고 **어떤 원리(How)로 구현되었는지**를 함께 서술합니다.

1.1. 문서 범위 및 구현 상태 표기

이 프로젝트는 현재 진행 중이며, 일부 기능은 설계만 되어 있고 아직 구현되지 않았습니다. 혼동을 막기 위해 장(章)별로 다음과 같이 구분합니다.

- 2 ~ 5장:** 현재 구현되어 동작하는 시스템 (5장은 알려진 한계 및 병목)
- 6장:** 향후 본/정찰 임무 **시나리오 설계** (LLM, 강화학습, HILS, 위험도 수식 등 미구현 항목 포함)
- 7장:** 노드·토픽 단위의 최종 상세 데이터 흐름(아키텍처 배선도)
- 8장:** 진행 중인 프로젝트의 모듈 분장 및 스프린트 일정

본문 각 항목에는 **[구현 완료]** / **[부분 구현]** / **[계획]** 라벨을 부여합니다.

1.2. 구현 현황 요약 (Implementation Status)

영역	기능	상태
인지	지형/장애물 분리, DBSCAN 클러스터링 + 센서 융합, YOLO + ByteTrack	[구현 완료]
판단	A* 전역 경로 + 동적 재탐색, APF 국소 회피(접선 척력 포함)	[구현 완료]
판단	맵 기반 위협(House002/Tank001) 회피 — 균일 25m 방사형 척력	[부분 구현]
판단	위협 시야각(FOV)·시선 차단(LoS) 기반 회피	[계획]
제어	비례 헤딩 제어 + 끼임 탈출 (PID 아님)	[구현 완료]
통신/시각화	HTTP 브릿지(W/A/S/D), RViz 마커	[구현 완료]
경로	A/B 정찰 루트 정의(routes.yaml)	[구현 완료]
데이터	정찰 로거(recon_logger) JSON 출력, 지형 굴곡(Pitch/Roll) 누적	[부분 구현] (노드 미연결)
시나리오	위험도/위협 지수 수식, 로컬 LLM 의사결정, HILS 스마트폰 연동	[계획]
로드맵	강화학습(RL)	[계획]

1.3. 관련 문서 (References)

- 빌드 및 실행 방법:** docs/ROS_파일들_실행_방법.txt , 루트 README.md
- 시뮬레이터 API 상세:** docs/SIMULATOR_API.md
- 실험 결과 및 계획:** docs/EXPERIMENT_RESULTS_AND_PLAN.md , docs/implementation_plan.md
- 패키지 구조 정리:** docs/README_REORGANIZED_STRUCTURE.md
- 통합/리팩터링 노트:** docs/INTEGRATION_NOTES_VISUAL_PERCEPTION.md , docs/REFACTOR_NOTES_LIDAR_CONFIG.md

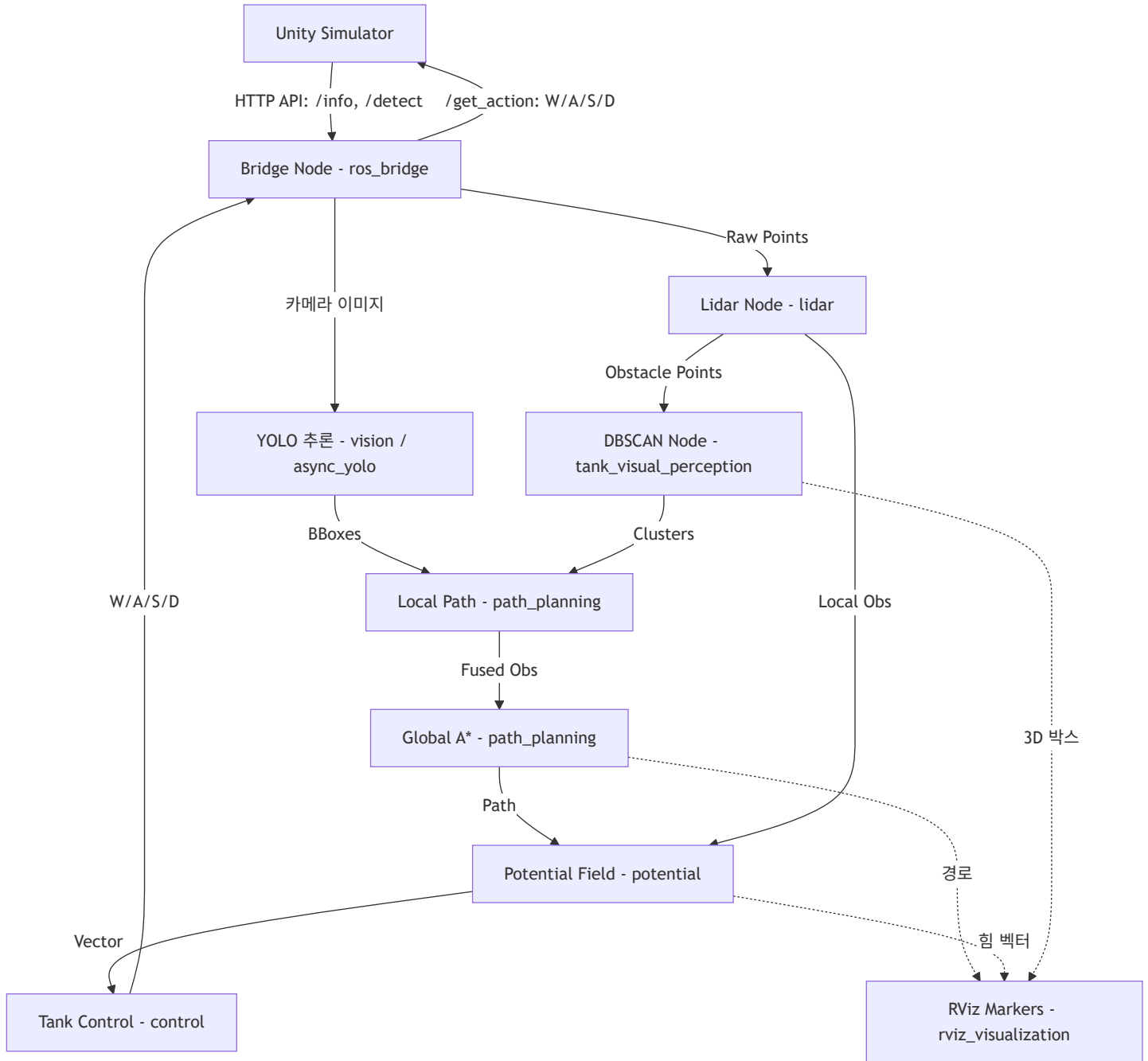
2. 전체 시스템 아키텍처 (System Architecture)

전차의 두뇌 역할을 하는 ROS2 시스템은 **8개의 독립 패키지**로 구성되어 시뮬레이터와 실시간으로 통신합니다.

ros_bridge , lidar , vision , tank_visual_perception , path_planning , potential , control , rviz_visualization

참고: 8장의 "8개 모듈"은 팀별 **태스크 분담 단위**(웹 대시보드, 강화학습 포함)로, 위의 8개 ROS2 패키지와는 다른 구분입니다. 숫자가 같은 것은 우연입니다.

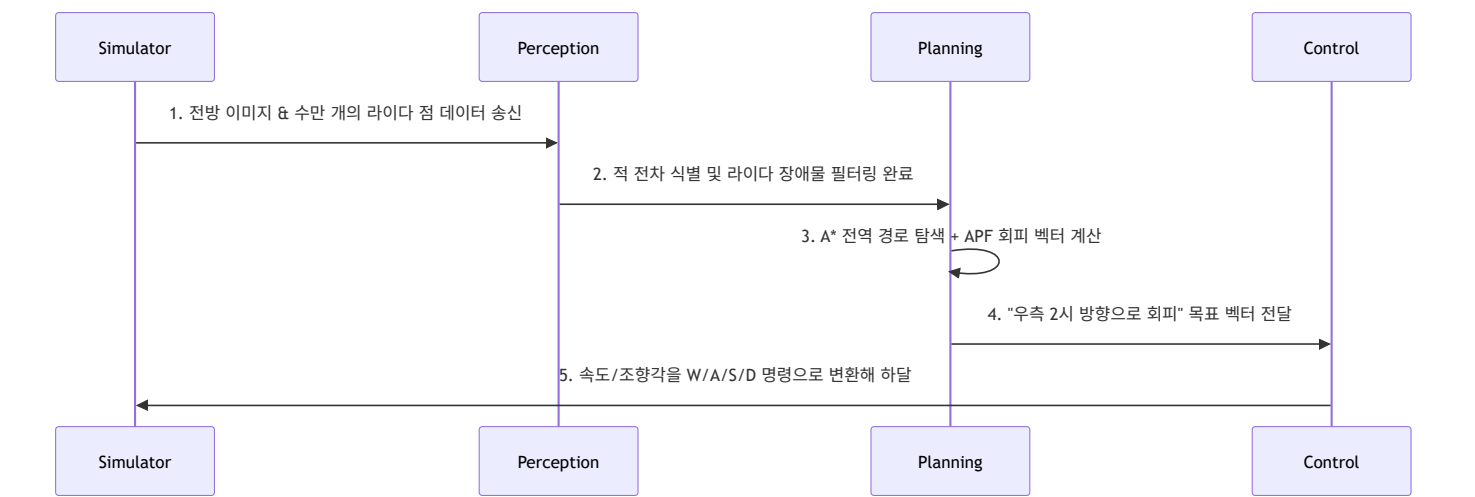
2.1. 패키지 구성 및 통신 구조 (다이어그램)



YOLO 노드에 대한 보충: YOLO는 별도의 독립 노드라기보다, 시뮬레이터가 `/detect` 엔드포인트로 보낸 이미지를 브릿지 내부의 비동기 워커(`async_yolo`) 또는 외부 추론 서버가 처리한 뒤 결과를 ROS2 토픽으로 발행하는 구조입니다.

위 그림은 개요입니다. 노드·토픽 단위의 정확한 최종 배선은 뒤쪽 **7장. 최종 상세 데이터 흐름**을 참고하세요.

2.2. 유기적 데이터 흐름 (1초의 시나리오)



2.3. 노드 및 토픽 인터페이스 (Interface)

패키지	실행 노드(executable)	핵심 역할	주요 입력	주요 출력
ros_bridge	ros_bridge	시뮬레이터 HTTP ↔ ROS2 중계	HTTP /info , /detect , /get_action	/tank/player/pose , /tank/ /tank/api/detect/* , /tank/perception/detectio
lidar	lidar_processor_node	지형/장애물 분리	라이다 원시 점(브릿지 경유)	/tank/sensor/lidar/detect
vision	yolo_debug_server	YOLO 추론/디버그 서버	카메라 이미지	YOLO 검출 결과
tank_visual_perception	lidar_dbscan_cluster_node , lidar_camera_overlay_node	DBSCAN 클러스터링, 라이다- 카메라 융합	/tank/sensor/lidar/detected_points_map , /tank/player/pose	/tank/visual_perception/l /tank/rviz/lidar_cluster_
path_planning	map_astar_planner_node , local_path_node	A* 전역 경로, 국소 융합/경로	클러스터, 목표 pose	전역 경로, /tank/path/looka
potential	potential_field_node	APF 국소 회피(접선 척력·위협 척력)	경로, 장애물, pose	/tank/local_target/pose (
control	tank_controller_node (노드명 tank_team_path_controller_node)	비례 헤딩 제어, 끼임 탈출	/tank/path/lookahead_pose , /tank/local_target/pose , /tank/player/state	/tank/control/command (W
rviz_visualization	rviz_visualizer_node 외 2종	RViz 3D 마커 렌더링	경로, 클러스터, 힘 벡터	visualization_msgs/Marker,

실행 파일명과 실제 ROS 노드명이 다른 경우가 있습니다(예: control 패키지의 실행 파일은 tank_controller_node 지만 노드명은 tank_team_path_controller_node). 디버깅 시 주의가 필요합니다.

2.4. 설정 오버라이드 (Configuration)

각 패키지의 임계값(격자 크기, 클러스터링 환경, 쿨다운 시간 등)은 config.py 에 하드코딩되어 있지 않고, 대부분 env_float / env_bool 을 통해 환경 변수(TANK_*) 나 ROS 파라미터로 재정의할 수 있습니다. 본문에 등장하는 수치는 모두 기본값이며, 재컴파일 없이 실험 단위로 튜닝하기 위한 구조입니다.

3. 핵심 구성 요소: "왜, 그리고 어떻게 작동하는가?"

이 장에서 다루는 기능은 모두 현재 구현되어 동작합니다. [\[구현 완료\]](#)

3.1. 지형(Terrain)과 장애물(Obstacle) 분리 로직 (lidar 패키지)

- **어떻게(How):** 수만 개의 레이저 점(Point Cloud)을 0.5m 단위의 격자(Grid)로 나눕니다. 각 격자의 가장 낮은 높이를 '현지 지면(Ground)'으로 삼고, 격자 안에서 0.4m 이상 갑자기 솟아오른 지점(Steep Cell)을 찾습니다. 그 격자 안에서 지면보다 0.2m 이상 높은 점들만 '부딪히는 장애물'로 분류하고, 나머지는 모두 '다닐 수 있는 지형'으로 묶습니다. (관련 코드: `lidar/terrain_utils.py`, 기본값은 `lidar/config.py`)
- **왜(Why):** 시뮬레이터는 이 점이 벽인지, 아니면 살짝 높은 언덕인지 알려주지 않습니다. 하지만 전차는 40cm 이하의 작은 바위나 언덕은 쉽게 타고 넘을 수 있습니다. 따라서 전차가 넘어갈 수 없는 진짜 수직 벽이나 거대한 장애물만 골라내기 위해 이러한 '높이 차이(Height Span)' 기반 알고리즘을 적용했습니다.

3.2. 점 데이터를 덩어리로 묶는 DBSCAN과 센서 융합 (tank_visual_perception 패키지)

- **어떻게(How):** 라이다로 골라낸 장애물 점들은 허공에 흩뿌려진 점들에 불과합니다. 이를 DBSCAN 알고리즘(반경 1.5m 이내에 2개 이상의 점이 있으면 하나의 덩어리로 묶음, `eps=1.5`, `min_samples=2`)을 통해 하나의 '객체(Object)'로 묶습니다. 이후 YOLO가 찾은 카메라 속 2D 박스와 라이다의 3D 덩어리를 하나로 합칩니다(Sensor Fusion). 라이다-카메라 융합과 클러스터링은 `tank_visual_perception` 패키지(`lidar_dbscan_cluster_node`, `lidar_camera_overlay_node`, `projection.py`)가 전담합니다.
- **왜(Why):** 점 자체만으로는 이게 건물인지, 나무인지, 적인지 알 수 없습니다. 점을 의미 있는 덩어리로 묶고 카메라의 시각 정보와 결합해야만, "이 방향 25m 앞의 이 덩어리는 적의 초소다"라는 구체적인 의미를 부여할 수 있습니다.

3.3. 접선 방향 반발력 (Tangential Repulsion - potential 패키지)

- **어떻게(How):** 인공 포텐셜 필드(APF)는 목적지가 전차를 끌어당기고, 장애물은 전차를 밀어내는 원리입니다. 여기에 '접선 방향 반발력'을 추가로 발생시킵니다. 장애물이 단순히 전차를 뒤로 밀어내는 것이 아니라, 장애물 표면을 따라 둥글게 미끄러지도록 밀어냅니다. (활성 구현: `potential_field_node.py`, `USE_TANGENTIAL_FORCE=True`)
- **왜(Why):** 일반적인 APF는 거대하고 넓은 벽 앞에서 수직 반발력을 받으면 좌우로 가지 못하고 멈춰버리는 맹점(Local Minimum)이 있습니다. 이를 해소하고 장애물 표면을 유연하게 타고 우회하도록 만들기 위한 로직입니다.

유지보수 주의: 같은 패키지의 `team_apf.py` 는 "접선 척력 삭제" 주석이 달린 **이전 버전(미사용 레거시)**입니다. 실제로 실행되는 노드는 접선 척력을 포함한 `potential_field_node.py` 입니다.

3.4. 동적 경로 재탐색 (Dynamic Repanning - path_planning 패키지)

- **어떻게(How):** A* 알고리즘으로 짜든 전체 경로(Global Path)를 계속 주시하다가, 라이다에서 발견된 거대한 장애물이 이 경로를 막아버린 것을 인지하면 일정 클라운(기본 8초)을 둔 뒤 A*를 처음부터 다시 실행합니다. (관련 코드: `map_astar_planner_node.py`, `DYNAMIC_REPLAN_COOLDOWN_SEC`)
- **왜(Why):** 출발 전 계획은 완벽하지 않으며, 전장에서는 예상치 못한 잔해나 벽이 길을 막을 수 있습니다. 무작정 들이받거나 제자리를 맴도는 것을 막고, 완전히 새로운 길을 개척하기 위한 유연성 확보 차원입니다.

3.5. 차량 고착 감지 및 탈출 (Stuck Escape - control 패키지)

- **어떻게(How):** 주행 중 5초마다 전차의 이동 거리를 측정합니다. 5초 동안 이동 거리가 1.5m에 미치지 못하면 '끼어서 갇혔다(Stuck)'고 판단합니다. 즉시 후진을 1.5초간 넣고, 제자리 회전(Pivot Turn)을 1.5초간 수행해 방향을 튼니다. (관련 코드: `tank_controller_node.py`, `control/config.py`)
- **왜(Why):** 물리 엔진 상 바위틈에 바퀴가 끼거나 경로가 꼬여 오도가도 못하는 상황이 발생할 수 있습니다. 인간의 개입 없이 스스로 함정을 빠져나가기 위한 핵심 페일세이프(Fail-Safe) 기능입니다.

3.6. 양방향 데이터 통신 브릿지 (ros_bridge 패키지)

- **어떻게(How):** 시뮬레이터가 보내는 원시 데이터(이미지, 라이다, GPS 등)를 HTTP API(Flask)로 받아 ROS2 토픽으로 변환하고, 역으로 자율주행 제어기가 내린 W/A/S/D 명령을 시뮬레이터로 전송합니다. (API 상세: `docs/SIMULATOR_API.md`)
- **왜(Why):** 유니티(C#) 환경과 ROS2(Python/C++) 환경은 통신 프로토콜이 다릅니다. 이 둘을 매끄럽게 연결하는 핵심 통로 역할을 하여 전체 시스템이 하나처럼 동작하게 합니다.

3.7. 시각 인지 및 딥러닝 추론 (vision 패키지)

- **어떻게(How):** 브릿지로부터 넘어온 전방 카메라 이미지를 YOLO 딥러닝 모델에 통과시켜, 화면 속 객체의 종류와 화면상 위치(Bounding Box)를 실시간으로 추출합니다. 현재 모델이 식별하는 클래스는 **rock(바위)**, **person(사람)**, **tank(전차)**, **tent(텐트)** 4종입니다. (`vision/config/yolo_detection.yaml`)

- **왜(Why):** 라이다(LiDAR)는 앞에 물체가 있다는 것은 알지만 그 정체는 모릅니다. "가로막은 것이 우회할 바위인지, 위협적인 적 전차인지"를 식별해야만 전술적 판단(강행 vs 복귀)을 내릴 수 있습니다.

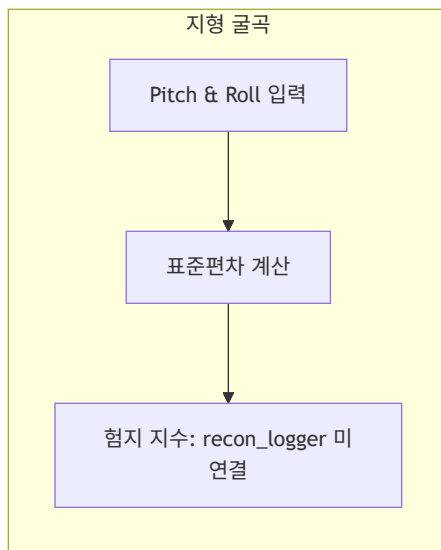
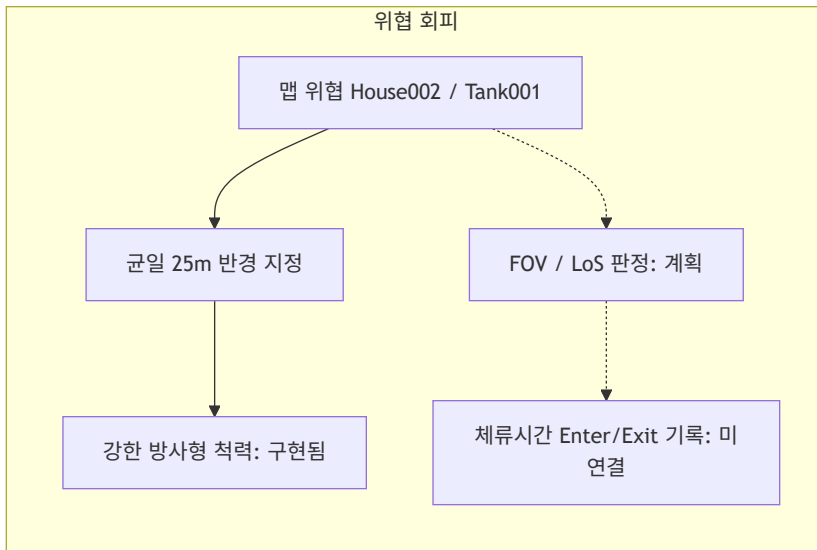
명칭 보충: 위협 회피 로직이 다루는 House002 (초소): Tank001 (적 전차)은 YOLO가 카메라로 식별하는 클래스가 아니라, **맵 정의 파일(ground-truth)에서 읽어 오는 별개의 정보원**입니다. YOLO의 tent / tank 클래스와는 출처가 다릅니다(4.2 참조).

3.8. 전장 상황 실시간 모니터링 (rviz_visualization 패키지)

- **어떻게(How):** 시스템 내부에서 숫자로만 처리되는 A* 탐색 경로, APF 반발력 화살표 벡터, 센서 융합으로 인식된 3D 장애물 박스를 모아 RViz2 화면에 3D 마커 (CUBE, ARROW, POINTS 등)로 렌더링합니다.
- **왜(Why):** 시뮬레이터 화면만으로는 전차가 현재 어떤 경로를 그리고 어떤 판단을 하는지 알 수 없습니다. 개발자와 사용자가 전차의 내부 의사결정 상태를 실시간으로 지켜보고 디버깅하기 위한 필수 관제창입니다.

4. 심화 정찰 및 위협 관리 시스템 (Advanced Analysis Systems)

이 시스템은 전차가 무사히 도착하는 것뿐 아니라, *****얼마나 들이지 않고, 얼마나 안전한 길로 왔는가*****를 정량적으로 평가하기 위한 설계입니다. 단, 아래에서 보듯 일부는 코드가 노드에 연결되지 않아 현재 데이터가 수집되지 않습니다.



4.1. 지형 굴곡(Terrain Roughness) 누적 분석 [부분 구현]

- 어떻게(How): 전차가 험한 길을 달릴 때 발생하는 차체의 흔들림(Pitch, Roll 각도)을 수집하고, 임무 종료 시 표준 편차를 계산해 험지 지수를 산출하도록 설계되어 있습니다. (recon_logger.py 의 log_body_angles , _calc_std)
- 현재 상태: 표준편차 계산 로직은 구현되어 있으나, 이를 호출하는 노드가 없어 실제로는 각도 데이터가 수집되지 않습니다. (5.4의 미연결 로거 문제와 동일한 뿌리)
- 왜(Why): 시뮬레이터에는 지형이 얼마나 울퉁불퉁한지에 대한 정답 맵이 없습니다. 따라서 전차가 실제로 측정한 진동을 역으로 활용해 험지 여부를 파악하고, A-B 루트 중 어느 쪽이 기동에 유리했는지 비교하기 위함입니다.

4.2. 적 위협 회피 및 시야 노출 측정 [부분 구현] / [계획]

- 현재 구현(동작): 위협 회피 노드(potential_field_node.py)는 맵 정의 파일에서 House002 · Tank001 위협의 좌표를 읽어와(parse_threats_from_map), 해당 위협 주변에 균일한 25m 반경의 강한 방사형 척력(THREAT_RADIUS=25 , K_THREAT=2000)을 발생시켜 전차가 그 반경 안으로 들어가지 않도록 크게 우회시킵니다. 위협 회피 자체는 기본 활성화되어 있습니다(USE_THREAT_AVOIDANCE=True).
- 아직 미구현(계획): 위협별 고유 시야각(FOV), 시선 차단(LoS) 판정, 시야 안으로 들어간 시점부터 빠져나올 때까지의 체류 시간(Dwell Time) 누적은 현재 활성화 경로에 적용되지 않습니다. FOV/LoS 로직은 미사용 코드(threat_detection.py , team_apf.py)에만 존재하며, 체류 시간 기록은 recon_logger (미연결)에만 들어 있습니다. 즉 위협은 방향과 무관한 원형 영역으로만 회피되며, 노출 시간은 아직 측정되지 않습니다.
- 왜(Why): 정찰 임무의 핵심은 '은밀함'입니다. 단순 충돌 회피를 넘어 적의 시야 반경 바깥으로 크게 우회해야 생존할 수 있고, 어쩔 수 없이 노출된 시간과 횟수를 기록해야 경로의 은밀성을 정량 평가할 수 있습니다. FOV/체류시간 측정은 이 평가를 완성하기 위한 향후 과제입니다.

5. 시스템 한계점 및 병목 현상 분석 (Limitations & Bottlenecks)

현재 시스템은 구조적으로 동작하고 있으나, 실험 규모가 커지거나 장시간 구동할 경우 프레임 드랍과 데이터 유실을 유발할 수 있는 병목과 버그를 안고 있습니다.

5.1. O(N²) 클러스터링 연산 부하 (LiDAR DBSCAN)

- 문제점: 클러스터링이 순수 파이썬 이중 for 문으로 점 사이 거리를 계산합니다. 포인트 개수(N)가 2500개면 매 프레임 약 625만(2500²) 번의 거리 계산이 필요해, CPU 부하가 급증하고 인지 파이프라인이 느려집니다. 해당 로직은 두 곳에 있습니다 — tank_visual_perception/lidar_dbscan_cluster_node.py (매 프레임 실행)와 lidar/payloads.py 의 cluster_lidar_points (obstacle_memory.py 가 누적 히스토리를 대상으로 호출).
- 참고: 전자는 max_points=2500 상한과 다운샘플링이 있어 포인트가 무한정 늘지는 않지만, 후자는 누적된 히스토리를 매번 다시 클러스터링하므로 시간이 지날수록 부하가 커질 수 있습니다. 추가로 이웃 확장 시 candidate not in seeds (리스트 멤버십 O(N)) 검사가 루프 안에서 반복되어 부하를 가중합니다.
- 해결 방안 제안: 해당 클러스터링 노드를 C++ ROS2 노드로 포팅하거나, 파이썬을 유지하되 scikit-learn / scipy 의 C 최적화 구현(KD-Tree 기반)으로 교체해 O(N log N) 수준으로 낮춰야 합니다.

5.2. 전역 경로 탐색의 메인 스레드 점유 (A* Blocking)

- 문제점: path_planning 의 메인 노드가 A* 연산을 별도 스레드로 분리하지 않고 ROS2 콜백 스레드 내에서 직접 실행합니다. (maybe_plan → plan_global_path 가 타이머 콜백에서 동기 실행되며, 노드에 스레딩/async가 전혀 없음)
- 결과: 경로를 새로 탐색하는 몇 초 동안 통신·제어 등 다른 기능이 '얼음(Freeze)' 상태가 되어, 동적 재탐색 중 제어 신호를 놓치고 벽에 부딪힐 위험이 있습니다.

5.3. 브릿지 노드의 과도한 객체 복사 (Deepcopy Overhead)

- 문제점: ros_bridge 가 수천 개의 좌표가 담긴 데이터를 파이썬의 무거운 deepcopy() 로 복사합니다(코드 전반 40여 곳). deepcopy는 주로 시뮬레이터가 호출하는 /info · /get_action HTTP 요청을 처리할 때마다 발생하며, 한 번의 요청 처리에 10여 회의 deepcopy가 일어납니다. 따라서 복사 빈도는 고정값이 아니라 시뮬레이터의 폴링 속도에 비례합니다. (별도의 publish_latest_state 타이머는 기본 1Hz)
- 결과: 빈번한 깊은 복사는 메모리 단편화와 CPU 캐시 소모를 유발하여 통신 지연(Latency)의 주범이 됩니다.
- 해결 방안 제안: 변경 부작용이 실제로 우려되는 경로에 한해서만 복사하고, 대용량 배열은 메모리 뷰/불변 참조로 전달하도록 개선이 필요합니다.

5.4. 데이터 유실 (Disconnected Logger 버그)

- 문제점: path_planning/recon_logger.py 에 정찰 데이터를 모아 JSON으로 출력하는 로거가 잘 짜여 있지만, 정작 어떤 노드도 이 로거를 호출(import)하지 않습니다.
- 결과: 주행 실험을 완료해도 보고서용 결과 JSON이 저장되지 않습니다. 영향 범위는 단순 JSON 저장에 그치지 않습니다 — 4.1 지형 굴곡, 4.2 체류 시간, 6.2 위험도 점수의 입력 데이터가 모두 이 로거에 모이도록 설계되어 있어, 로거가 연결되지 않은 현재는 이 측정들 자체가 일어나지 않습니다. 정찰/평가 파이프라인을 완성하려면 메인 주행 노드에 로거를 배선하는 작업이 선행되어야 합니다.

5.5. [참고] 정지된 장애물을 YOLO가 트래킹(Tracking)하는 이유

- **의문점:** 바위나 텐트 같은 장애물은 제자리에 있는데 왜 YOLO의 Track Mode(ByteTrack)를 켜서 객체를 추적할까요?
- **원리:** 장애물은 가만히 있지만, **카메라를 단 전차가 계속 이동(steady_state_speed 최대 약 19.45, 대략 시속 20km 수준)하기 때문에 화면 속에서는 장애물이 끊임없이 움직입니다.** 트래킹을 켜면 흔들리는 화면 속에서도 0.1초 전의 바위와 현재의 바위를 '동일한 객체(Track ID)'로 묶어, 노이즈(오인식)를 걸러내고 일관된 회피 기동을 보장합니다. (`tracking.enabled: true` , `tracker: bytetrack.yaml`)

5.6. [참고] 추가로 식별된 코드 이슈 (개선 백로그)

아래는 위 병목과 별개로 코드 검토 중 확인된 항목입니다. 문서 정확도와 직접 관련된 일부는 본문에 이미 반영했고, 순수 코드 결함은 별도 개선 과제로 관리합니다.

- **위험 회피 코드 3중 중복:** 활성 `potential_field_node.py` (균일 25m 척력) 외에 미사용 `team_apf.py` ($\pm 45^\circ$ 콘)·`threat_detection.py` (FOV 60°, LoS)가 공존하고 `parse_threats_from_map` 이 두 곳에 중복됩니다. 4.2 혼선의 근본 원인 — 레거시 정리 또는 FOV 로직 통합 필요.
- **제어기 방식:** 현재 제어기는 헤딩 오차에 비례한 조향(P 제어)만 사용하며 적분/미분(I-D) 항이 없습니다. 8장에서 언급한 'PID 제어기'는 향후 목표입니다.
- **소규모 결함:** 컨트롤러가 `max_speed` (약 19.45)를 읽지만 실제 W 명령 가중치에 반영하지 않음, 클러스터 거리 필드명 `nearest_origin_distance_m` 이 실제로는 전차 기준 거리를 담음(명칭 불일치), 끼임 탈출 피벗이 항상 우회전("D")으로 고정됨.

6. 정찰 및 본 임무 시나리오 명세 (Scenario Design) [계획]

본 장은 **향후 시나리오 설계**입니다. 아래 항목 중 [계획]으로 표시된 것(위험도/위험 지수 수식, 로컬 LLM 의사결정, HILS, 강화학습)은 **아직 구현되지 않았습니다.** 단, 6.1의 A-B 루트 정의(`routes.yaml`)는 구현되어 있습니다.

본 섹션은 사전 정의된 A, B 루트를 바탕으로 정밀 정찰을 수행하고, 분석된 위험도를 근거로 최적 루트로 본 임무를 수행하는 전체 시나리오의 흐름과 판단 기준을 정의합니다.

6.1. 사전 전제 조건 (Pre-condition)

- **사전 정보:** 상공 드론 정찰로 목적지의 대략적 위치와 거시적 접근 루트 2개(A, B)의 존재만 파악된 상태입니다.
- **미지수:** 각 루트 내부의 미세 지형, 세부 장애물 위치, 적 초소(House, Enemy Tank)의 정확한 위치와 시야 범위는 모르는 백지 상태(Blind)에서 출발합니다.
- **루트 정의 [구현 완료]:** A-B 루트의 경유 좌표는 `path_planning/config/routes.yaml` 에 정의되어 있고 `route_loader.py` 로 로드됩니다.
- **주행 기반 [계획 가정]:** 현재는 A*(전역)와 APF(국소) 기반으로 주행하며, 추후 이 모듈을 **강화학습(RL)** 모델로 교체할 수 있도록 모듈화한다는 것을 가정합니다.

6.2. [시나리오 1] 정밀 정찰 및 루트 평가 (Precision Reconnaissance) [계획]

- **임무 목표:** A-B 루트를 각각 직접 주행하며 맵 세부 정보를 수집하고, 그 데이터를 바탕으로 본 임무에 투입할 '최적 루트'를 선정합니다.
- **수행 흐름:** 출발지에서 목적지까지 A 루트를 따라 자율주행하며 장애물과 초소를 매핑합니다. 도착 후 시뮬레이터를 초기화(Restart)하고 B 루트를 주행합니다.
- **위험도(Risk Score) 산출 기준 [계획]:** 아래 수식은 설계안이며 아직 구현 코드가 없습니다. 입력 데이터(발견 개수, 노출 시간, 우회 횟수, 굴곡도 등)는 5.4의 `recon_logger` 에 모이도록 설계되어 있으나, 로거가 연결되기 전까지는 산출이 불가능합니다.

Risk Score = (W1 × 적/초소 발견 개수) + (W2 × 적 시야 노출 시간) + (W3 × 우회 횟수 및 이탈 거리) + (W4 × 지형 굴곡도) + (W5 × 소요 시간)

- **적 및 초소 발견 개수:** 루트 상의 적 전차/초소의 절대 수 (직접적 치명 위험)
- **적 시야 노출 시간:** 탐지 반경 내 체류 시간
- **우회 횟수:** 장애물이 많아 우회한 횟수 (기동 지연 및 위험 노출 증가)
- **지형 굴곡도:** 차체 흔들림(Pitch/Roll)에 따른 기동성 저하 요인
- **로컬 LLM 방식 [계획]:** 위 정량 데이터를 텍스트 프롬프트로 묶어 Qwen/Gemma 모델에 전달해, **"B루트가 험난해 고장 확률은 있지만 적에게 들키는 것보다는 나으므로 은밀성을 우선한다"**와 같은 유연한 판단을 기대합니다. (현재 LLM 연동 코드는 없습니다.)

6.3. [시나리오 2] 본 임무 투입 및 돌발 상황 대처 (Main Mission) [계획]

- **임무 목표:** 시나리오 1에서 선정한 최적 루트로 은밀하게 침투하며, 돌발 위험 발생 시 즉각적인 생존 판단(강행 vs 복귀)을 내립니다.
- **수행 흐름:** 정찰 때 파악한 적 시야 밖으로 크게 우회합니다. 주행 중 정찰 때 없던 새로운 '정적 팝업 장애물'이나 '고정형 적 전차'가 경로를 가로막습니다.
- **돌발 상황 의사 결정 기준 [계획]:** 피격이나 체력(Health) 시스템이 없으므로, 수학적 '동적 위험 지수'를 산출해 임계값을 넘으면 복귀, 넘지 않으면 강행하는 방식을 기본(Standard)으로 구현하는 것을 목표로 합니다. (현재 위험 지수 산출 코드는 없습니다.)

Dynamic Threat Score = (W1 × 팝업 객체의 고유 위험도) + (W2 × 우회에 필요한 추가 예상 거리) + (W3 × 우회 시 예상 적 시야 노출 시간) - (W4 × 목 적지 근접도)

- **팝업 객체의 고유 위험도:** 식별 객체 종류(바위, 적군, 전차, 초소). 단순 바위(낮음) vs 시야를 가진 적/초소(높음)
- **추가 예상 거리:** 우회로를 찾았을 때 늘어나는 거리. 우회로가 없거나 너무 멀면 복귀 확률 증가

- 적 시야 노출 시간: 우회 경로가 적 탐지 반경을 가로지를 때의 페널티
- 목적지 근접도 (감산 요인): 목적지가 코앞(예: 10% 미만)이면 위험 지수를 크게 깎아 강행 유도

[심화 시도: LLM 대체 연구] [계획]

기본 수식 구현 후, 위 4개 변수 값을 그대로 로컬 LLM(Qwen 등) 프롬프트로 전달해, 수식 대신 AI가 직접 "Return"/"Continue"를 판단하도록 로직을 대체할 수 있는지 추가로 시도합니다.

- 결과 행동: 산출된 위험 지수가 낮으면 우회하여 [강행], 너무 높으면 목적지를 출발지로 스왑하여 **[복귀]**합니다. (LLM 도입 성공 시 LLM 응답값으로 행동을 결정.)

6.4. HILS (Sim-to-Real) 기반 모바일 센서 연동 [계획]

시나리오 2의 돌발 상황을 소프트웨어적 생성에 그치지 않고, 스마트폰을 활용한 HILS(Hardware-in-the-Loop Simulation) 브릿지로 구현하는 것을 목표로 합니다. (현재 관련 구현 없음.)

1. 스마트폰 카메라 인식 (Real-to-Sim Obstacle): 실제 환경에서 스마트폰 카메라로 현실 객체(모형 바위, 전차 등)를 비추면 이를 YOLO로 인식해 ROS2로 전송합니다. 유니티 화면에 3D 모델로 띄우지는 못하지만, ROS2 내부 지도(Costmap)와 장애물 메모리에 해당 좌표를 주입합니다. 결과적으로 유니티에서는 전차가 허공을 피하는 것처럼 보이지만, ROS2 두뇌 입장에서는 현실의 모형을 회피 기동합니다.
2. 스마트폰 IMU 센서 연동 (Real-to-Sim Roughness): 스마트폰 IMU(가속도계, 자이로) 데이터를 실시간으로 ROS2로 스트리밍해, 물리적 흔들림을 가상 전차의 지형 굴곡도(Roughness) 수치에 즉각 반영합니다.

6.5. 시나리오 논리적 빈틈 및 보완책 (Loopholes & Solutions) [계획]

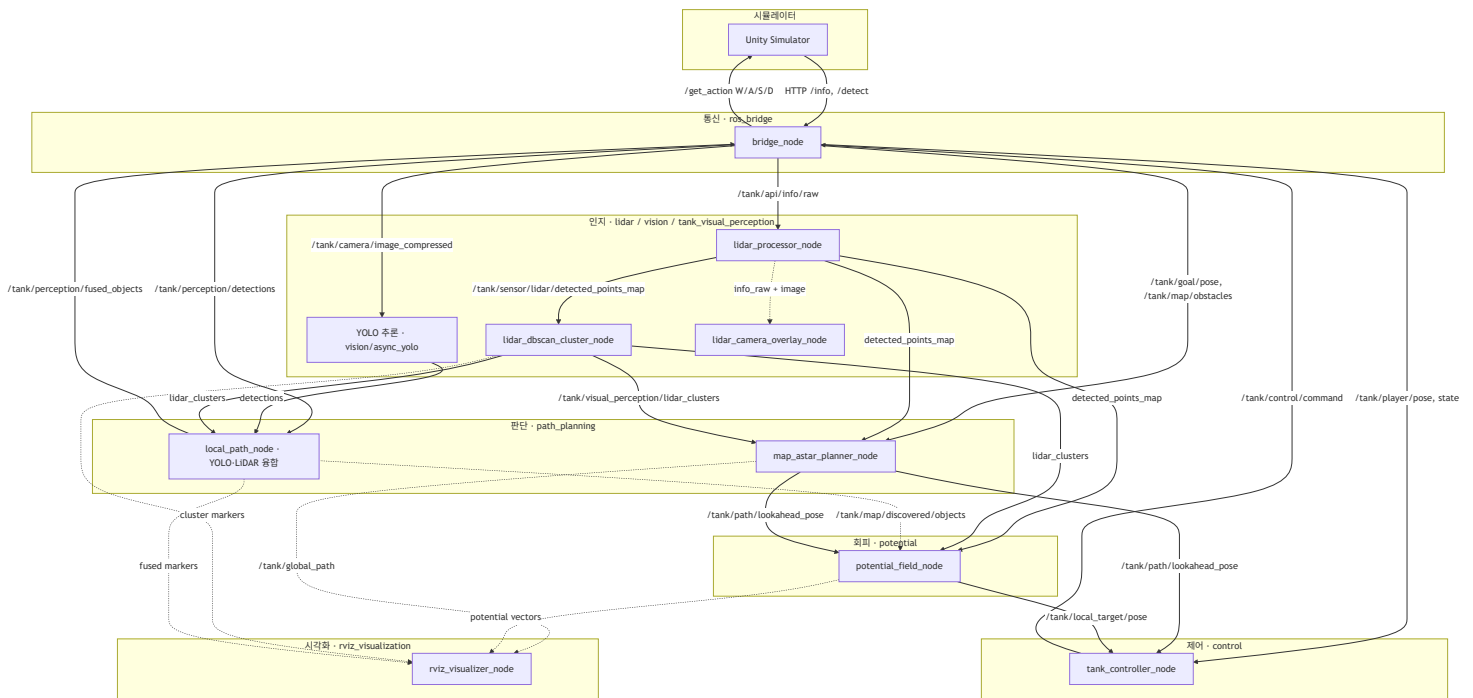
1. 스마트폰-시뮬레이터 좌표계 동기화 (Calibration): 핸드폰 카메라가 비추는 현실 좌표를 유니티 맵에 맞추기 위해, 시나리오 시작 시 스마트폰 화면 기준점을 전차의 현재 위치(Origin)로 영점 조절하는 초기화 단계가 필요합니다.
2. 의사결정 지연으로 인한 충돌 (LLM Latency): 로컬 LLM이 결정하는 데 1~3초가 걸릴 수 있습니다. 이 시간 동안 전차가 이동해 충돌하는 것을 막기 위해, 위험 감지 순간 **[긴급 제동(Emergency Brake)]**을 걸어 판단할 시간을 확보합니다.

6.6. 고급 기술 연동 원리 및 로드맵 (LLM & RL) [계획]

- 로컬 LLM 연동 원리: ROS2 노드가 현재 상황(남은 거리 50m, 적 발견 등)을 숫자로 뽑아 프롬프트 문자열을 만들고, Ollama 서버(Qwen/Gemma)에 HTTP로 전송합니다. LLM이 {"decision": "Return"} 형태의 JSON을 반환하면 파이썬 코드가 이를 파싱해 목적지 좌표를 스왑합니다.
- 강화학습(RL) 도입 로드맵: 한 번에 대체하는 것은 불가능하므로 3단계로 도입합니다.
 - 1단계 (제어 대체): A*·APF가 방향(Vector)을 주면, 그 방향으로 가는 W/A/S/D 조작만 RL이 담당.
 - 2단계 (APF 대체): A* 전역 경로만 보고, 국소 장애물 회피를 APF 대신 RL이 담당.
 - 3단계 (End-to-End): A*·APF를 모두 제거하고, 센서 데이터(LiDAR, 카메라)와 목적지 좌표만으로 스스로 길을 찾고 회피.

7. 최종 상세 데이터 흐름 (Detailed Architecture & Data Flow)

2.1이 "한눈에 보는 개요"라면, 본 장은 실제 코드의 pub/sub 토픽을 그대로 옮긴 최종 배선도입니다. 모든 토픽명은 2.3 인터페이스 표 및 각 패키지 config.py의 TOPIC_* 상수와 일치합니다.



(실선 = 핵심 제어 흐름, 점선 = 보조/시각화 흐름. /tank/player/pose · /tank/goal/pose 등 포즈 계열은 다수 노드에 브로드캐스트되므로 가독성을 위해 대표 경로만 표기했습니다.)

계층별 설명

- 시뮬레이터 ↔ 브릿지 (ros_bridge):** Unity가 HTTP로 센서·이미지를 보내면 브릿지가 이를 안정적인 ROS2 토픽으로 변환해 팬아웃하고, 역방향으로 제어기의 W/A/S/D 명령을 /get_action 응답으로 하달합니다. /tank/player/pose , /tank/player/state , /tank/goal/pose , /tank/map/obstacles 는 여러 노드가 공통으로 구독합니다.
- 인지 (lidar · vision · tank_visual_perception):** lidar_processor_node 가 지형/장애물을 분리해 /tank/sensor/lidar/detected_points_map 을 발행합니다. lidar_dbscan_cluster_node 가 이 점들을 객체 단위로 묶어 /tank/visual_perception/lidar_clusters 를 만들고, vision 의 YOLO가 /tank/perception/detections 를 내놓습니다. lidar_camera_overlay_node 는 라이더-카메라 융합을 보여주는 디버그 영상을 생성합니다.
- 판단 (path_planning):** local_path_node 가 YOLO 박스와 LiDAR 클러스터를 융합해 /tank/perception/fused_objects 와 발견 객체 (/tank/map/discovered/objects)를 산출합니다. map_astar_planner_node 는 전역 경로 (/tank/global_path)와 추종용 /tank/path/lookahead_pose 를 발행하며, 장애물이 경로를 막으면 동적으로 재탐색합니다(3.4 참조).
- 회피 (potential):** potential_field_node 가 A*의 lookahead_pose 를 추종 목표로 삼아, 목적지 인력 + 장애물 척력 + 접선력 + 위험 척력을 합성한 결과를 /tank/local_target/pose (및 /tank/potential/*_vector)로 발행합니다.
- 제어 (control):** tank_controller_node 가 목표를 /tank/Local_target/pose (APF) → /tank/path/lookahead_pose (A) → /tank/goal/pose * 순으로 선택하고, 비례 헤딩 제어로 W/A/S/D를 산출해 /tank/control/command 로 브릿지에 보냅니다. 브릿지가 이를 시뮬레이터로 전달하며 루프가 닫힙니다.
- 시각화 (rviz_visualization):** rviz_visualizer_node 가 경로·클러스터·융합 객체·포텐셜 벡터를 구독해 RViz2 3D 마커로 렌더링합니다.

페루프 요약: 각 노드는 독립 타이머(Hz)로 비동기 동작하며, 시뮬레이터 → 브릿지 → 인지 → 판단 → 회피 → 제어 → 브릿지 → 시뮬레이터 순환이 매 주기 반복됩니다.

보조 노드: static_map_loader_node (정적 맵 코스트 로드)와 terrain_record_finalize_node (지형 기록 마무리)는 RViz/기록 보조용으로 메인 제어 루프 밖에 있습니다.

8. 프로젝트 모듈 분장 및 애자일 스프린트 일정

팀원들이 병목(Blocking) 없이 병렬로 개발할 수 있도록, 시스템을 8개의 독립 태스크 모듈로 나누고 멘토링 데드라인에 맞춘 애자일(Agile) 스프린트 일정을 설계했습니다.

참고: 여기서의 "8개 모듈"은 팀 태스크 분담 단위입니다. 일부(모듈 2 웹 대시보드, 모듈 8 강화학습)는 아직 ROS2 패키지로 존재하지 않으며, 2장의 8개 ROS2 패키지 구분과는 다릅니다.

8.1. 병렬 개발을 위한 모듈별 태스크 분류 (Task Breakdown)

- **[모듈 1] 시스템 & 통신 (System & Bridge):** 무거운 배열 복사(5.3)를 메모리 뷰 형태로 최적화. 시나리오 1 종료 시 환경을 초기화하는 Restart API 등 전체 State Machine 흐름 제어 로직 작성.
- **[모듈 2] 웹 관제 화면 (Web Dashboard):** React/HTML 등으로 실시간 카메라 영상, 위험도 점수, A* 미니맵, LLM 판단 텍스트를 한눈에 보는 프론트엔드 구축. 7/28 발표 시각 자료의 핵심.
- **[모듈 3] 인지 - LiDAR 최적화 (Perception):** NumPy, scikit-learn 등을 사용한 벡터화 연산으로 이중 for문 $O(N^2)$ 병목(5.1) 해결.
- **[모듈 4] 인지 - Vision & HILS (Perception):** YOLO 객체 정확도 튜닝 및 스마트폰 카메라로 비춘 모형의 bounding box 좌표를 ROS2로 스트리밍하는 HILS 연동 앱 구축.
- **[모듈 5] 판단 - 전역 & 국소 경로 (Global & Local Planning):** A* 비동기 스테딩(5.2) 구현. 팝업 장애물 대응 APF 가중치 튜닝. 복귀(Return) 시 목적지를 출발지로 스왑하는 로직 통합.
- **[모듈 6] 차량 제어 (Vehicle Control):** Planning 결과를 전차 조향(W/A/S/D)으로 바꾸는 제어기. 현재는 헤딩 오차 비례(P) 제어이며, 안정화를 위해 PID로 확장하는 것이 과제. LLM 판단 시간을 버는 '긴급 제동' 및 '끼임 탈출' 로직 포함. (더미 벡터로 독립 테스트)
- **[모듈 7] 데이터 로깅 & LLM 통합 (Logging & Intelligence):** recon_logger.py 를 노드에 연결(5.4)하고 IMU 굴곡도 로직 작성. 로깅된 JSON을 로컬 Ollama(Qwen) 프롬프트로 보내 강행/복귀를 판단하고 결과를 파싱하는 브릿지 전담.
- **[모듈 8] 강화학습 환경 구축 (Reinforcement Learning):** Unity ML-Agents 툴킷을 세팅하고 탱크 에이전트 C# 스크립트로 관측(Observation)·보상(Reward) 구조를 짜는 기초 환경 구축 (향후 로드맵 목적).

8.2. 마일스톤 및 멘토링 대비 스프린트 계획 (Timeline)

개발 마감일(7/10)과 최종 발표일(7/28) 사이의 3회 멘토링(6/20, 7/4, 7/18)에 맞춰 가시적 성과를 도출합니다.

- **Sprint 1: 개별 모듈 독립 구현 (6/13 ~ 6/19)**
 - 목표: **1차 멘토링(6/20)**용 각 모듈 단독 시연 자료 완성. (예: LiDAR 파이션 최적화 속도 비교, 웹 관제화면 UI 뼈대 등)
- **Sprint 2: 부분 통합 및 시나리오 1 완성 (6/21 ~ 7/3)**
 - 목표: 모듈 부분 통합 및 **2차 멘토링(7/4)**에서 시나리오 1(정밀 정찰 및 맵핑) 전체 흐름 데모.
- **Sprint 3: 최고난도 통합 및 개발 마감 (7/5 ~ 7/10)**
 - 목표: 시나리오 2(시물 팝업 + 스마트폰 HILS) 및 로컬 LLM 통합.
 - [7/10 Code Freeze]: 치명적 버그 수정 외 신규 기능 추가 전면 금지.
- **Sprint 4: 시연 영상 녹화 및 검증 (7/11 ~ 7/17)**
 - 목표: 시나리오 1, 2 구동 영상 녹화·편집. **3차 멘토링(7/18)**에서 최종 영상 연출 및 품질 검증.
- **Sprint 5: 최종 산출물 및 발표 준비 (7/19 ~ 7/27)**
 - 목표: 종합 보고서 완성, 아키텍처 다이어그램 및 PPT 마감. 발표 모의 연습 후 7/28 최종 발표.